Introduction

Biomedical imaging plays a pivotal role in modern healthcare, enabling clinicians to visualize and diagnose various medical conditions. Among these conditions, pneumonia is a significant global health challenge; it is the leading infectious cause of death among children under 5, killing approximately 700,000 children a year. Quick and accurate diagnosis of pneumonia is important for improving patient outcomes. The use of artificial intelligence techniques, particularly convolutional neural networks, has revolutionized the field of biomedical imaging analysis by offering automated tools for disease detection.

Convolutional neural networks (CNNs) have demonstrated success in various computer vision tasks, particularly in image classification. CNNs can learn important features from raw image data without manual feature engineering, making them ideal for biomedical image analysis. For pneumonia detection, CNNs have potential to help diagnostic efficiency by automatically extracting relevant features from chest radiographs. AI techniques can also uncover subtle patterns and features that humans may not be able to see. An automated model can assist healthcare providers and ultimately lead to better patient outcomes and more efficient healthcare.

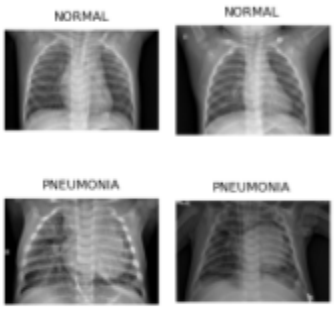
We will demonstrate the performance and efficacy of CNNs in classifying chest X-ray images for pneumonia detection using our own experimental results. Through this research, we aim to contribute to the ongoing efforts to utilize AI technologies to improve in the world of healthcare.

Background

By leveraging Python-based machine learning libraries such as TensorFlow, Keras, and OpenCV, we developed a CNN model that can accurately distinguish between normal and pneumonia-affected chest X-rays. Convolutional neural networks represent a specialized class of deep learning models, used for image classification tasks. Unlike traditional neural networks, CNNs exploit the spatial structure present in images through a hierarchical architecture of convolutional and pooling layers. This architectural design allows them to automatically learn relevant features from raw pixel data, making them particularly suitable for computer vision tasks.

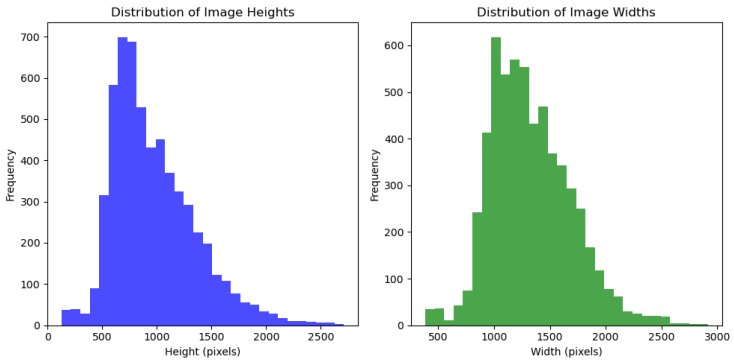
A fundamental aspect of CNNs is their ability to learn hierarchical representations of data. For pneumonia detection from chest X-ray images, CNNs offer a promising approach because they can discern subtle differences in lung texture and structure. Pneumonia manifests as areas of opacification, or cloudiness, within the lung parenchyma, which can be visualized on X-ray images. However, distinguishing between pneumonia and normal lung tissue can be challenging, especially when it is in the early stages or subtle. CNNs have demonstrated the potential to overcome these challenges by automatically extracting discriminative features from X-ray images and classifying them as pneumonia-affected or normal.

Methodology



Dataset Examples

Our dataset consisted of chest X-ray images selected from pediatric patients of ages one to five years old from Guangzhou Women and Children's Medical Center. All chest radiographs were screened for quality control by removal of all low quality scans as well as diagnoses graded by three expert physicians. This dataset was designed to be able to be used as training for AI models like ours, so there were no unlabeled images or bad quality images that we had to remove. Every image in the dataset is a projection radiograph of the chest with clear labels of either "NORMAL" or "PNEUMONIA". The dataset is organized into 3 folders (train, test, val) and contains subfolders for each image category (Pneumonia/Normal), with a total of 5,863 X-Ray jpeg images. The training set is composed of 5216 images, the test set 624 images, and the validation set 23 images. The dimensions of the images varied. Below, there are histograms depicting the distribution of image height and width in this dataset. The maximum image height was 2713 pixels and the minimum was 127 pixels. The maximum image width was 2916 pixels while the minimum was 384 pixels. In the training set, there are 1341 images classified as "normal" and 3875 classified as "pneumonia". In the test set, there are



234 images classified as normal and 390 classified as “pneumonia”.

Our data preprocessing begins with our “ImageDataGenerator” instances. To start off, we normalized our data with our “rescale” parameter that scales the pixel values of the images to the range [0,1]. Next, we performed some data augmentation on our training set when we set our zoom_range parameter to specify the range for randomly zooming images in and out. We also set vertical_flip to True, which enables randomly vertically flipping images. Augmenting the data increases variability in our data which will help our model generalize better and prevent overfitting. Lastly, we resized our images by setting our target size to (img_dims, img_dims), where img_dims is a parameter we define as (150, 150, 3). This resizes all our images to 150 by 150 pixels and represents color with RGB channels. This preprocessing pipeline ensures that our data is properly prepared for training our model.

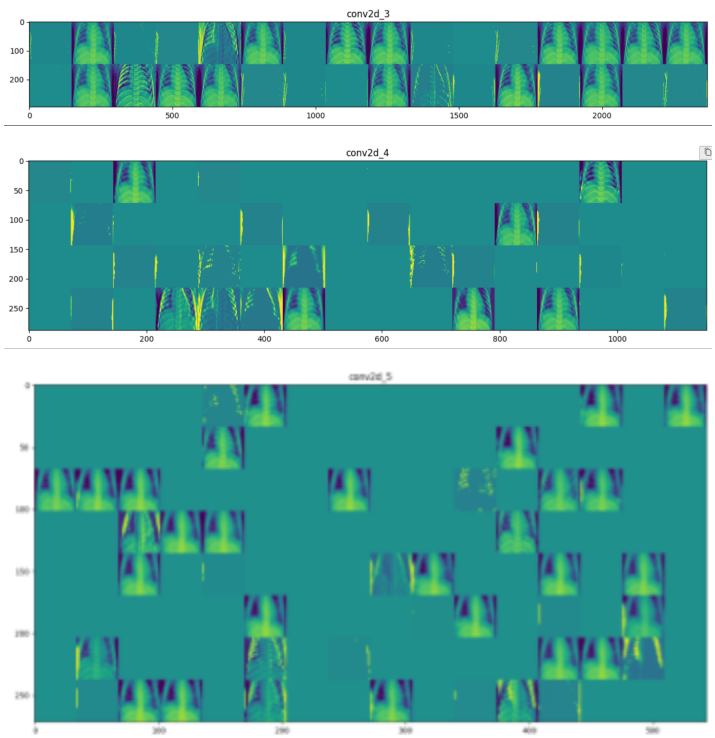
CNN architecture typically begins with an input layer followed by a sequence of convolutional layers, where each layer applies filters to extract features from the images. Our CNN model consists of 9 layers and uses the sequential model from Keras, providing us with a flexible framework to adjust convolutional and pooling layers. Convolutional layers skim over input images with a sliding window approach, applying filters within this window to extract features such as edges, textures, or shapes from the images. Each filter generates a feature map that represents the presence of specific features at various locations in the image. We applied 3 convolutional layers with kernel sizes (3,3). The Kernel size of 3x3 is recommended (and used in famous models such as VGG, ResNet, and GoogLeNet) because there are fewer parameters compared to other sizes such as 5x5, 7x7, etc. This means that there's less information for the network to store, making it less likely to memorize the training data and more likely to learn general patterns, resulting in less overfitting. We used the approach of increasing complexity with each layer, so the first Conv2D layer has 32 filters to capture low-level features such as edges, colors, and gradients (direction and magnitude of the most significant change in intensity or color across pixels). In the second and third Conv2D, the filter is doubled each time to capture higher-level features. This practice of doubling is a conventional approach used in many published CNN models. We also pair each convolutional layer with a pooling layer, to decrease computational load and enhance feature detection.

The max-pooling layer reduces the spatial dimensions of the feature maps created by the convolutional layer. Max pooling works by the same sliding approach as the convolutional layer but has a different functionality: it slides a 2x2 filter across the feature map and takes the maximum value within the window as the output for the new, reduced feature map. The result is a new feature map that is smaller in size, which preserves the most prominent features detected from the previous layer. Switching back and forth between the convolutional and max-pooling layers has proven to be very efficient. Doing this iteration allows the network to systematically discard less relevant features and emphasize more distinct patterns at each stage, and therefore the network becomes better and better at identifying the essential characteristics of the input with increasing precision and minimizing redundancy along the way. This, combined with the application of the Rectified Linear Unit (ReLU), was a key factor in the success of AlexNet, a highly efficient CNN model designed by Geoffrey Hinton and two of his students that demonstrated remarkable performance in image classification tasks.

After our convolutional and pooling layers, we use a layer to flatten the output and prepare it for classification. The flattened layer takes the multi-dimensional output of the preceding convolutional and pooling layers and "flattens" it into a one-dimensional vector for the input of the subsequent dense layers. Flattening is necessary to transition from the convolutional layers to the fully connected layers, as the fully connected layers require one-dimensional input. The second-to-last layer is a fully connected layer with 128 neurons and ReLU activation that transitions from feature extraction to pattern recognition, allowing the model to understand and interpret the spatial hierarchies and features detected by the convolutional layers. The output of this layer is a 128-dimensional vector that represents a high-level, abstracted form of the input image that we can then pass to the final layer. The final dense layer has a single neuron and sigmoid activation; it produces the final output of the model: the binary classification of “NORMAL” or “PNEUMONIA”.

The Rectified Linear Unit (ReLU) function is significant in our model, as it is applied to each feature map following the convolution operation. It plays a critical role in mitigating the vanishing gradient problem that arises during the backpropagation process. The vanishing gradient problem occurs as the derivatives of activation functions such as sigmoid and tanh become increasingly small, particularly for large positive or negative inputs. This phenomenon

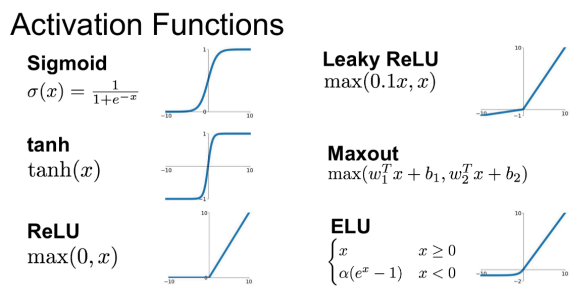
hampers the training process, as the gradients diminish significantly as they are back-propagated through the network layers, making it challenging for the network to learn effectively from the training data. The ReLU activation function offers a simple yet effective solution to this problem. By providing a derivative of either zero for negative inputs or one for positive inputs, ReLU ensures that the gradients remain consistent and do not diminish too much during backpropagation. This results in more stable and efficient training of deep neural networks, letting them effectively learn complex features and patterns from the input data. Furthermore, the simplicity of the ReLU function results in faster computation and its piecewise linear nature also allows for easier interpretation of network behavior and facilitates optimization processes. However, this characteristic of the ReLU also comes with a drawback. To visualize this, we included the feature map after each of our 3 convolutional layers:



Feature Maps

The dying neuron problem occurs when neurons in a neural network are not being activated. In the CNN, we can observe this in the feature maps below as blank or inactive regions, indicating that certain filters are not responding to variations in the input data. The root cause of this issue is tied to the activation functions described in the ReLU: operating on the principle, mentioned earlier, of setting negative inputs to zero. If the neuron is consistently outputting a negative value, it gets stuck - never being activated, hence the name: ‘dying’ neuron. We looked at multiple different techniques to prevent this issue for further exploration of this project. The dying neuron problem occurs during the forward pass due to the activation functions being called then, and the impact of the issue is realized fully during backpropagation. As we are adjusting the neural network weights based on the error gradient calculated from the output layer back towards the input layer during

the backpropagation process, the gradients will be zero for the neurons that output zero during the forward pass of our training. So, during this backpropagation, those neurons' weights will never be updated because their error gradient is zero - which is ‘killing’ the neuron, and the model will exclude neurons that could have contributed to the learning process as they remain inactive.



The main alternatives to ReLU, we have looked into to advance our project are Leaky ReLU, Parametric ReLU (PReLU), or even completely different activation functions like ELU (Exponential Linear Unit) or SELU (Scaled Exponential Linear Unit). These functions are specifically designed to allow a positively small gradient when the activation of the neurons would be zero, which keeps the gradient flow during backpropagation and significantly reduces the risk of the ‘dead’ neuron. For the negative inputs, leaky ReLU produces a small positive output which is proportional to the input. The effect of this is that during backpropagation, there will still be a gradient through the neuron for negative inputs, which is different from the normal ReLU where the gradient would be zero. This prevents the neurons from becoming inactive (‘dying’). This will be implemented for scaling up our project, and we assume it will lead to a higher accuracy rate.

Results

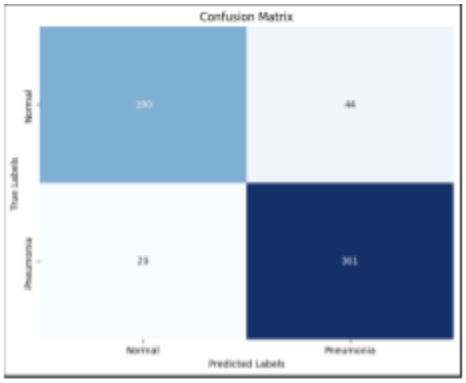
Following training, we evaluated the model's performance using the test data, calculating performance metrics such as accuracy, precision, recall, and F1 score. Additionally, we constructed a confusion matrix and generated a classification report to provide a comprehensive assessment of the model's performance. Our model exhibited promising performance as it was able to achieve an overall accuracy of about 0.88 and a test loss of 0.33. Our confusion matrix and classification report further support the potential of our model. Important metrics that stand out are the precision of

Classification Report :

	precision	recall	f1-score	support
Normal	0.87	0.81	0.84	234
Pneumonia	0.89	0.93	0.91	390
accuracy			0.88	624
macro avg	0.88	0.87	0.87	624
weighted avg	0.88	0.88	0.88	624

Classification Report

the Normal class which was at 0.87 and the Pneumonia class at 0.89 which was based on the correctly predicted percentage of each of their respective classes, as well as the recall score which we can see is 0.81 for the Normal class and 0.93 for the Pneumonia class. The model had a higher recall for the "pneumonia" class compared to the "normal" class, indicating that it is better at correctly identifying pneumonia cases. This can be seen in our confusion matrix, with 361 images correctly classified as Pneumonia. These performance metrics indicate strong performance which is on the right track for practical applicability to real-world scenarios.

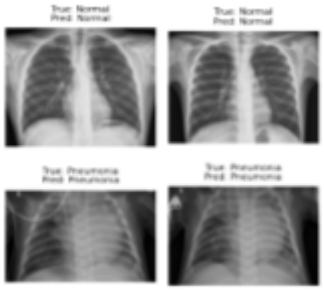


Confusion Matrix

We also made a loss graph and an accuracy graph to compare our training data and validation data through the epochs. Looking at our accuracy plot, we can see that as the model trains over time, it becomes increasingly adaptive at fitting the training data. For most epochs, the training and validation accuracy lines are within 5% of each other except for Epoch 5 which had a major decrease in validation by 10%. We think this may be due to some overfitting and some adjustments in the model complexity need to be made. The same can be said for the loss graph where the overall decreasing loss line indicates that the model's predictions are getting closer to the true labels with each epoch. The spike in the validation loss occurs on the same epoch as the validation accuracy which is an increase in the error rate of your model when evaluating on the validation dataset.

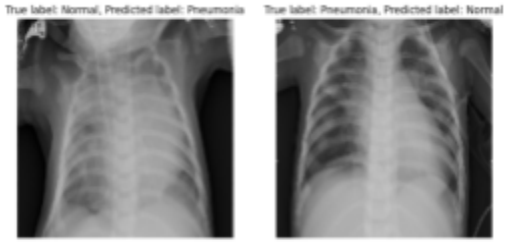
Training and Validation Accuracy Plot

Training and Validation Loss Plot


Correctly classified images

Given our accuracy of 0.88, it's evident that while our CNN model performs well overall, there are still some misclassifications. By examining general examples of correctly classified images, we can see that pneumonia often presents with abnormal clouding

resulting in our model accurately identifying such cases. However, we also have misclassified examples where the cloudy appearance may lead to a prediction of pneumonia, despite the true label being normal. This shows the complexity of distinguishing between normal and pneumonia-affected lung tissues, particularly when subtle abnormalities are present. Further analysis of these misclassifications can provide valuable insights for refining our model and improving its performance in pneumonia detection.



Misclassified examples

Discussion

With a relatively high accuracy for classifying chest X-rays as pneumonia-affected or normal, our model could be used for real-world applications. A model such as ours can lead to improved patient outcomes and prevent complications associated with delayed diagnosis. This could also contribute to public health efforts, as pneumonia can be effectively detected early on and efforts can begin earlier to mitigate the spread of it within communities. This project was a valuable experience that helped deepen our knowledge and understanding of machine learning techniques after many hours of hard work and group collaboration for the amount of time and effort required to handle data preprocessing and model training. Working with CNNs opened our eyes to how much data processing is needed and it's clear now that improving how we handle data processing and model-making is critical for better performance and efficiency. Building and structuring different layers of the CNN helped us understand the layering that goes into a convolutional neural network. We learned the time and computing power it takes to build up CNN, especially on a single computer and no access to higher % RAM power through cloud computing subscriptions. One challenge was different software package merging errors, where we had to update Keras and TensorFlow libraries, as we were not able to run both newest versions simultaneously.

Regarding our model, we found that some limitations occurred that should be further looked into. Our dataset was very limited, as it only consisted of young children from a pediatric hospital in

China. For further research, a good goal would be to utilize a dataset consisting of chest radiographs from people of all ages and regions. Another limitation was our small validation set that only consisted of 23 images. This potentially limits the ability to effectively tune hyperparameters and optimize the model's architecture. In the future, we would want to use a larger validation set or set aside images with the train-test split method from the training data to use for validation. Additionally, we would like to increase the accuracy of the CNN model by using some of AlexNet's overlapping pooling & dropout techniques. We would also consider expanding our approach to include other models, such as logistic regression, to further explore the efficacy of different methodologies. We also found that for the dying neuron problem, we are looking to integrate 'leaky ReLU' to solve this issue and ideally achieve a higher accuracy.

Conclusion

Our project was to build a Convolutional Neural Network used for biomedical image classification that specifically focused on pneumonia detection in young children. We took a dataset from Kaggle that contained train, test, and validation data images which we preprocessed using an image generator. Our CNN architecture consisted of convolutional layering and max-pooling which was able to accurately predict between normal and pneumonia chest x-rays at about 87% accuracy. This indicated strong, promising results for real-world application in particular to medical imaging within healthcare. Through future efforts and research, our goal is for this model to serve as a useful tool to improve medical diagnostics by detecting signs of pneumonia and potentially other diseases to help healthcare around the world.

Contributions

Wilson Cam - Worked on code that included preprocessing techniques, building the CNN model, and evaluation plots. Did the Results, Discussion, and Conclusion sections for the presentation and written report.

Frederick Kiessling - Model architecture, built & presented details on each layer (& choice of parameters), research on ReLU inclusion, programmed feature map outputs to visualize

convolutional layer's outputs, read research papers on models such as AlexNet for model design, and explained ReLU optimization & limiting the 'dying neuron' problem.

Aimee - Worked on preprocessing the data, exploratory data analysis, parts of the model architecture, large portion of written report, introduction and EDA for presentation.

Reference

Dataset: https://data.mendeley.com/datasets/rscbjbr9sj/2